

## Lab 5 - Building AI Agents with LangChain

### Overview

The lab focuses on building structure AI workflow using LangChains and Agents. The primary objective was to move beyond single-prompts interactions and design multi-step reasoning systems that combines LLMs with structure logic and external tools.

The lab has 3 sections:

Single Prompt Chain

Multi-step Sequential Chain

Agent with Tools (dynamic reasoning and actions)

I implemented controlled pipelines as well as autonomous agent behavior and analyzed how decision-making differs between static and dynamic agents.

### Implementation Details

#### 1. Single Prompt Chain

The first task involved creating LLM pipeline using:

Prompt Template

Chat OpenAI

Pipe operator to chain operator

System operates: Prompt → LLM → Response

This exercise also demonstrates that prompts can be separated from models to increase their flexibility.

The main idea:

Decoupling prompts from models can increase their flexibility.

#### 2. Multi-Step Sequential Chain (Customer Review Pipeline)

The second method caught my attention because of its strategy. The strategy is to divide the problem into two steps:

1. Sentiment Analysis
2. Response Generation based on the determined sentiment

I developed two different prompt templates. Two LLM calls were made. The input of the first call was used in the second call. This is similar to real-world decision-making processes, where reasoning is segmented rather than packed into a single prompt.

What I found out:

- Breaking down tasks into smaller components helped me understand things better.
- The performance of sentiment classification was more stable when tested separately.
- The quality of reasoning was slightly reduced when all the tasks were packed together in a prompt.
- The debugging of multi-step chains was easier compared to lengthy prompts.

The main idea:

The cognitive burden on the model is reduced when tasks are segmented into sequential chains.

#### 3. Agent with Tools (Unit Conversion Agent)

This final and crucial part dealt with the creation of an AI agent using the pieces we had developed so far:

`initialize\_agent`, a `Tool`, `ConversationBufferMemory`, and an `LLM` with reasoning capabilities. Unlike the rigid chains we had developed so far, the agent would be able to:

- Select the tool it would apply
- Perform calculations on the fly
- Remember the conversation as it progressed
- Think through the intermediate results

In the unit conversion problem, the agent would parse the user's statement, apply the appropriate tool, and produce the

final result.

This marked a significant shift from the static chains to an autonomous reasoning system.

### Observations on Agent Performance

#### What went well

- Defined prompts ensured that the selection of tools was a perfect match.
- Memory ensured smooth and continuous conversations.
- Agents demonstrated better performance in multi-step reasoning than single-step chains of reasoning.
- Clearly written tool descriptions ensured better decision making.

#### What did not work perfectly well

- Ambiguous tool descriptions caused agents to behave erratically.
- Agents sometimes over-explain simple calculations.
- Agents' memory sometimes retains unnecessary context.
- Debugging tools is harder than debugging simple prompts.

#### Chains vs Agents Comparison

| Feature              | Chains  | Agents    |
|----------------------|---------|-----------|
| Control              | High    | Moderate  |
| Flexibility          | Limited | High      |
| Debugging            | Easy    | Harder    |
| Autonomy             | Low     | High      |
| Multi-step reasoning | Manual  | Automatic |

The distinction that I could easily identify is that chains are deterministic because their execution path is predetermined. On the other hand, agents provide more variability because of the reasoning step that determines what to do next in real time. The more variability that agents provide is offset by the fact that you might see a slight difference in output even when you provide similar inputs.

#### What we learned

- Chains perform best in a deterministic step-by-step process.
- Agents perform best in a dynamic decision environment.

### Key Takeaway

This lab session made one major concept stand out to me:

Structure makes for intelligent systems.

#### By Combining:

- Prompt Engineering, Memory, Tool Integration, Sequential Logic

An Intelligent Reasoning Agent.

Another major concept that was made apparent was the impact that the creation of the prompt had on the tools that were selected. The wording of the prompt had a direct impact on the reasoning process for the action to be taken.

Modular design was another major takeaway from the lab. Rather than creating a large and potentially brittle prompt, breaking the problem into smaller components made for more robust behavior.

As a systems designer, the addition of agents brings flexibility but also introduces a level of unpredictability.

### Conclusion

What this lab did demonstrate is that intelligent AI isn't created with large models; it requires orchestration. The beauty of chains is control and predictability, and the beauty of agents is the ability to be flexible and independent. The difficulty is finding the right balance between the two. The art of creating good AI systems requires the right balance in prompt engineering, defining the tools, memory, and effectively evaluating the reasoning process. The transition from single prompts to workflow is the distinction between using a model and creating AI systems.